

Vânătoare de comori

Limită de timp: 8 s Limită de memorie: 256 MiB

Cauți o comoară pierdută de mult timp pe un câmp uriaș de celule de dimensiune $N \times N$, folosind o busolă magică. Există în total K cufere cu comori ascunse în diferite celule ale câmpului. Scopul tău este simplu: să le găsești pe toate!

Când așezi busola pe una dintre celule, aceasta va găsi cel mai scurt drum către un cufăr cu comori folosind numai mișcări la stânga, sus, dreapta și jos (adică spre cel mai apropiat cufăr conform distanței Manhattan). Apoi va afișa direcția primei mișcări. Dacă există mai multe prime mișcări valide care duc către cel mai apropiat cufăr (sau către mai multe dintre ele), busola le va returna pe toate. Dacă această celulă conține o comoară, busola va indica acest lucru în schimb.

Ori de câte ori găsești un cufăr cu comori, îi poți goli conținutul, dar nu poți elimina cufărul — este prea greu. Busola ta nu știe dacă un cufăr este plin sau gol. Ea va indica drumul către cel mai apropiat cufăr, indiferent dacă este gol sau plin.

Încearcă să găsești locația tuturor cuferelelor cu comori folosind cât mai puține interogări ale busolei.

Cerință

Aceasta este o problemă interactivă. În fiecare test (adică la fiecare execuție a programului tău), programul va trebui să rezolve mai multe vânători de comori. Trebuie să interacționezi cu evaluatorul folosind o bibliotecă furnizată de organizatori. Această bibliotecă conține următoarele declarații:

- **void** *NextHunt*(**int** & N , **int** & K) — apelează această funcție pentru a începe următoarea vânătoare de comori. Ea va returna dimensiunea grilei în parametrul N și numărul de comori în K . Dacă nu mai există alte vânători de comori de rezolvat în execuția curentă, funcția va seta N și K la -1 ; în acest caz, trebuie să închei programul cu codul de ieșire 0. Reține că poți apela această funcție înainte de a găsi toate comorile din vânătoarea curentă, de exemplu dacă încerci să obții doar punctaj parțial.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — acestea sunt constantele folosite în valorile returnate de funcția *Query* (vezi mai jos).
- **int** *Query*(**int** x , **int** y) — dacă celula cu coordonatele (x, y) conține o comoară, această funcție returnează *TREASURE*. În caz contrar, returnează suma uneia sau mai multor constante dintre *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* și *DIR_DOWN*, indicând ce mișcări returnează busola atunci când este plasată pe celula (x, y) . Coordonatele x și y trebuie să fie numere naturale între 0 și $N - 1$. (Notă: în această problemă, coordonatele y cresc de sus în jos.)

După ce *NextHunt* setează $N = K = -1$, programul tău nu mai trebuie să apeleze nici *NextHunt*, nici *Query*. De asemenea, nu trebuie să apeleze *Query* înainte de primul apel al lui *NextHunt*. Dacă programul nu respectă aceste restricții, sau dacă efectuează mai

mult de 1000 de interogări într-o singură vânătoare de comori, sau dacă furnizează valori pentru x și/sau y în afara intervalului permis atunci când apelează *Query*, biblioteca va termina execuția programului și va returna verdictul run-time-error (RTE) pentru cazul de test curent.

O comoară este considerată găsită dacă ai interogat celula care o conține cel puțin o dată. Dacă programul tău apelează *NextHunt* înainte de a găsi toate comorile din vânătoarea curentă, acest lucru nu este considerat o eroare, dar îți va afecta scorul (mai multe detalii despre punctaj mai jos).

Pentru a accesa biblioteca, programul tău trebuie să includă fișierul header `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Poți descărca acest fișier header de aici: `treasurehuntlib.h`.

Pentru a te ajuta în dezvoltarea soluției, este disponibilă aici o implementare simplă a bibliotecii: `treasurehuntlib-public.cpp`. Pentru a o compila împreună cu programul tău, este suficient să adaugi numele fișierului care conține implementarea funcțiilor din header ca parametru al comenzii de compilare, de exemplu:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

dacă `foo.cpp` este numele fișierului care conține soluția ta.

Implementarea din `treasurehuntlib-public.cpp` conține, pe lângă funcțiile menționate anterior, o altă funcție cu antetul `void InitFromFile(const char *fileName)` care citește o listă de vânători de comori și vă permite să simulați un joc cu acestea în locul unora random. Veți găsi mai multe detalii în fișierul `treasurehuntlib-public.cpp`.

Pe serverul de evaluare va fi folosită o implementare diferită a bibliotecii, așa că nu trebuie să faci presupuneri despre modul exact în care funcționează implementarea. Poți însă presupune că nu poluează spațiul global de nume cu alte declarații decât cele enumerate mai sus (*NextHunt*, *Query* și cele cinci constante).

Codul tău nu trebuie să citească de la intrarea standard și nici să scrie la ieșirea standard, deoarece acestea vor fi folosite de implementarea noastră a bibliotecii de pe serverul de evaluare pentru a comunica cu restul mediului de evaluare.

Intrare

Restricții

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- În cadrul unei singure execuții a programului tău vor exista cel mult 100 000 vânători de comori.
- În cadrul unei singure vânători de comori poți efectua cel mult 1000 de interogări.
- Sistemul de evaluare nu este adaptiv.

Subtaskuri

- Subtask 1 (10 puncte) $K = 1$
- Subtask 2 (30 puncte) $K = 2$
- Subtask 3 (60 puncte) $K = 3$.

Punctaj

Un subtask poate consta din mai multe cazuri de test (mai multe execuții ale programului tău), iar fiecare caz de test poate conține mai multe vânători de comori. În scopul punctării, toate vânătorile de comori ale unui anumit subtask sunt evaluate împreună, indiferent de modul în care au fost distribuite inițial între cazurile de test. Pentru a i -a vânătoare de comori, notăm dimensiunea grilei cu $N_i \times N_i$, numărul de comori cu K_i , numărul de interogări efectuate de programul tău cu Q_i și numărul de comori găsite cu F_i . De asemenea, notăm cu S numărul total de puncte disponibile pentru acest subtask. Atunci, numărul de puncte acordat programului tău pentru acest subtask este:

- Dacă programul tău a găsit întotdeauna toate comorile (adică dacă $F_i = K_i$ pentru toate valorile lui i), acesta primește

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f\left(\frac{Q_i}{\max\{1, \lceil \log_2 N_i \rceil\}}\right) \text{ puncte, unde } f(t) = \begin{cases} 1, & t \leq 11 \\ 1 - (t - 11)/9, & 11 \leq t \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Dacă programul tău nu a găsit întotdeauna toate comorile (adică există un i pentru care $F_i < K_i$), acesta primește

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ puncte.}$$

Cu alte cuvinte, primești jumătate din puncte pentru găsirea tuturor comorilor și cealaltă jumătate pentru găsirea lor folosind cât mai puține interogări. Pentru un scor perfect, soluția ta trebuie să găsească toate comorile în cel mult $11 \lceil \log_2 N_i \rceil$ interogări. Între $11 \lceil \log_2 N_i \rceil$ și $20 \lceil \log_2 N_i \rceil$ interogări, scorul scade liniar; iar dacă soluția ta efectuează mai mult de $20 \lceil \log_2 N_i \rceil$ interogări, va primi doar prima jumătate a punctelor pentru găsirea tuturor comorilor. (Simbolurile $\lceil \cdot \rceil$ înseamnă că valoarea lui $\log_2 N_i$ este rotunjită în sus la cel mai apropiat număr întreg.)

Dacă formulele de mai sus produc un număr real de puncte pentru subtask, acesta va fi rotunjit la cel mai apropiat număr întreg.

Dacă programul tău produce o eroare de execuție sau nu respectă protocolul descris mai sus pentru utilizarea bibliotecii, va primi 0 puncte pentru întregul subtask. Pentru a obține puncte parțiale pentru găsirea doar a unui subset al comorilor, programul tău trebuie, prin urmare, să încheie căutarea în mod corespunzător prin apelarea funcției *NextHunt*.

Exemplu

Apel	Valoare returnată
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN \mid DIR_RIGHT$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$